Informe del proceso – 21 Black Jack (duelo contra el computador)

Mini Proyecto 1 · Fundamentos de Programación Orientada a Eventos (FPOE, 750014C) · Universidad del Valle · Profesora: Dra. Paola Johanna Rodríguez Carrillo

1. Datos del equipo y acceso al proyecto

Integrantes	**Nicolás Revelo Noreña** · código 2537974 · nicolas.revelo@correounivalle.edu.co · GitHub `NicolasRevelo` · Persona B (vista y controlador) **Douglas Velez Monginaite** · código 2516737 · douglas.velez@correounivalle.edu.co · GitHub `douglasvelez-crypto` · Persona A (modelo)
Repositorio	https://github.com/NicolasRevelo/Mini-proyecto-1-FPOE
Versión entregada	Commit evaluado: [pegar aquí el enlace al commit final, p. ej. .../commit/<hash>]
Acceso de la docente	Usuario de GitHub de la profesora agregado como colaborador del repositorio: [usuario de la profesora]

2. Análisis y organización del trabajo

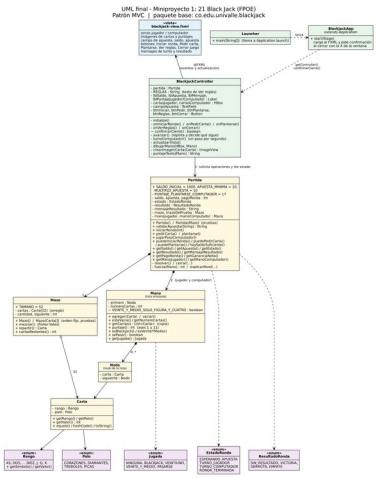
Partimos del enunciado, de las 12 historias de usuario (HU01 a HU12, con criterios de aceptación numerados) y de la rúbrica. Con esos tres documentos, y con apoyo de Claude, armamos un plan de 7 días, repartimos el trabajo en dos roles y escribimos un contrato (`CONTRATO.md`) con los nombres de clases y métodos que acordamos entre los dos; cualquier cambio a ese contrato se habla y se sube con un commit. **Douglas (A)** hizo el modelo: `Partida`, `Mazo` (arreglo de 52 cartas), `Mano` (lista enlazada) y `Nodo`. **Nicolás (B)** hizo la vista y el controlador: el FXML, `BlackjackController`, las imágenes de las cartas, el UML, la matriz de pruebas y el README.

Para que B no esperara al modelo, primero se hizo una `Partida` provisional con tres escenarios fijos y la pantalla se desarrolló contra ella; luego se reemplazó por el modelo real sin cambiar el controlador. Cronología según el historial de commits:

- **23 oct:** proyecto base JavaFX con Maven; imágenes de las 52 cartas, clases provisionales y UML inicial.
- **26 y 27 oct:** HU01 (pantalla inicial), HU03 (cartas del jugador) y HU04 (pedir carta); registro de IA al día.
- **28 y 29 oct:** Douglas entrega el modelo real. El 29 se integra al proyecto, y se agregan plantarse y el turno del computador (HU05, HU06, HU08), las reglas y el cierre con confirmación (HU11, HU12), la matriz de pruebas, las pruebas JUnit, el README y el UML final.

3. Decisiones de diseño y cambios realizados

- **MVC con responsabilidades claras.** `Partida` decide el estado, las reglas y los pagos. `BlackjackController` nunca cambia el saldo, las manos ni el estado: le pide cosas a la `Partida` y repinta con `actualizarVista()`. Los botones se habilitan con `puedeIniciarRonda()`, `puedePedirCarta()` y `puedePlantarse()`, así la regla "fuera de turno no se puede" vive en un solo lugar.
- **Estructuras que exige el enunciado.** `Mazo` usa un arreglo de 52 posiciones, se mezcla con Fisher-Yates y reparte sin repetir. `Mano` guarda las cartas en una lista enlazada de `Nodo`. `Mazo` acepta un orden fijo de cartas, lo que permite probar manos exactas (As + 10, K + 9 + 5).
- **Turno del computador paso a paso.** `jugarPasoComputador()` hace un solo paso (repartir, pedir o resolver) y el controlador lo llama con una `PauseTransition` de 1 s. Así se ve la secuencia, no se congela la ventana y se cumple el criterio 6.6.
- **Prioridad de jugadas** en un método `fuerza()`: Blackjack > Veintiuno > Veinte y medio > resto. Pagos: 2,5 veces con Blackjack, el doble en victoria, devolución en empate.
- **Veinte y medio** = dos primeras cartas suman 14 (criterio 7.5), con un interruptor en `Mano` por si la docente prefiere "figura + 4" del enunciado. Con 14 el computador pide, porque 6.4 dice que pide con menos de 17.
- **Cambios en el camino:** el controlador pasó a anunciar Veinte y medio, Blackjack y la falta de fichas (HU04 4.3, HU07 7.5, HU10 10.4); se integró el



UML final (`docs/UML_final.png`)

modelo real y se eliminó una copia anidada del proyecto; `.idea` salió del repositorio; el `pom.xml` fija `release 21` y la versión de Surefire para JUnit 5.

4. Dificultades, pruebas, colaboración y aprendizajes

Dificultades. (a) El proyecto no compilaba con "modules are not supported in -source 8": el `pom.xml` usaba Java 8 y se cambió a `release 21`. (b) Tras un pull volvió a aparecer un JDK "corretto-17" inexistente porque la configuración de `.idea` de un compañero se había subido; se dejó de versionar `.idea`. (c) El repositorio tenía una copia anidada del proyecto y un `Mazo.java` vacío en la raíz. (d) Pedir carta parecía funcionar pero no estaba conectado, y las jugadas especiales no se anunciaban; son los **dos registros de error y corrección** del anexo. (e) En la revisión final se encontró una marca de conflicto de merge dentro del registro de IA y se eliminó.

Pruebas. (1) Matriz de pruebas de la interfaz, con 55 casos en los cinco bloques de la rúbrica (apuestas y saldo, mazo y rondas, puntajes y resultado, turnos y rival, interacción). En la última ronda los 55 cumplieron y no se observaron fallas; las manos poco frecuentes se forzaron con un mazo de orden fijo temporal, que no se subió al repositorio. (2) 83 pruebas JUnit 5 del modelo (`CartaTest`, `ManoTest`, `MazoTest`, `PartidaTest`): valores y ases, Blackjack, Veintiuno, Veinte y medio, pasarse, validación de apuestas, reglas del computador, resultados y pagos, acciones fuera de turno y 2000 rondas aleatorias sin cartas repetidas ni saldos negativos. Resultado de `mvn test`: [confirmar: 83 pruebas, 0 fallas].

Trazabilidad: de la historia de usuario a la prueba.

HU	Qué pide	Dónde se implementa	Dónde se prueba
HU01	Pantalla inicial y botones por estado	`blackjack-view.fxml`, `actualizarVista()`	Matriz P01, P54, P55
HU02	Validar la apuesta	`Partida.validarApuesta()`	Matriz P02–P11; `PartidaTest`
HU03	Iniciar ronda y repartir al jugador	`Partida.iniciarRonda()`, `Mazo`	Matriz P12, P18–P23; `MazoTest`
HU04	Pedir carta	`onPedirCarta()`, `Partida.pedirCarta()`	Matriz P39–P41; `PartidaTest`
HU05	Plantarse	`onPlantarse()`, `Partida.plantarse()`	Matriz P42–P43; `PartidaTest`
HU06	Turno del computador	`turnoComputador()`, `jugarPasoComputador()`	Matriz P44–P47; `PartidaTest`
HU07	Puntajes y jugadas especiales	`Mano.puntaje()`, `getJugada()`	Matriz P24–P30; `ManoTest`
HU08	Resultado de la ronda	`Partida.resolver()`, `fuerza()`	Matriz P31–P38; `PartidaTest`
HU09	Liquidación de la apuesta	`Partida.cerrar()`	Matriz P13–P17; `PartidaTest`
HU10	Jugar otra ronda	`Partida.iniciarRonda()`, `avanzar()`	Matriz P21–P22; `PartidaTest`
HU11	Consultar las reglas	`onVerReglas()`	Matriz P49–P50
HU12	Cerrar con confirmación	`confirmarCierre()`, `BlackjackApp`	Matriz P51–P53

Colaboración. El historial tiene 19 commits propios de Nicolás entre el 23 y el 29 de octubre, por historia de usuario, más los de Douglas ([número de commits de Douglas; sale de git shortlog -sn]). El contrato `CONTRATO.md` fue el punto de acuerdo entre vista y modelo. Revisión cruzada: Nicolás revisó el modelo de Douglas (14 casos con mazo fijo y 20.000 rondas simuladas) y se corrigieron los textos de los mensajes; Douglas [qué revisó del controlador].

Aprendizajes. Un botón habilitado no está conectado hasta probarlo. Una afirmación de la IA, o nuestra, no vale hasta que se ejecuta o se compara con los criterios de aceptación. Y separar reglas (modelo) de presentación (controlador) permitió trabajar en paralelo y cambiar el modelo sin tocar la pantalla.

5. Declaración de uso de IA e índice de evidencias

Usamos **Claude (Anthropic)** entre el 22 y el 29 de octubre de 2026, los dos integrantes. Esta declaración se consolida por equipo y sale del registro `docs/REGISTRO_IA.md`, donde se anotó cada uso en su momento. No se usaron imágenes ni código de terceros: las cartas las dibuja un script propio (`docs/generar_cartas.py`).

Uso	Quién	Qué aportó la IA	Qué cambiamos o descartamos	Cómo se verificó
Plan de 7 días, contrato A–B y partida provisional	Nicolás	Plan, `CONTRATO.md` y 8 clases con 3 escenarios fijos	Se cambió `plantarse()` para que el computador juegue paso a paso	Se compiló y se recorrió una ronda completa por consola
Entorno: IntelliJ, Git, JDK 21 y `pom.xml`	Nicolás	Pasos guiados; `release 21` en el compilador	Se adaptó al proyecto real	Abrió la ventana de ejemplo; luego la del juego
52 imágenes de cartas y UML inicial	Nicolás	Script Python de dibujo y diagrama	Sin cambios	Se revisó una hoja con todas las cartas: valor y palo coinciden con el nombre
Controlador y vista (HU01–HU06, HU08, HU11, HU12)	Nicolás	Código propuesto y explicado paso a paso	Nicolás lo tecleó, integró y probó; se descartó una indicación errónea de dónde pegar métodos	Se ejecutó cada historia y se llenó la matriz de pruebas
Partida y Mazo: dos cartas iniciales, nivel de jugadas, `mezclar`, interruptor de Veinte y medio	Douglas	Un `for`, una cadena de `else`, un Fisher-Yates y un `switch`	Douglas repitió el comando en vez del `for`, simplificó el código de `mezclar` y cambió sus variables	[completar con lo que Douglas verificó]
Revisión e integración del modelo	Nicolás	14 casos con mazo fijo, simulación de 20.000 rondas y limpieza de mensajes	Solo textos (espacios, tildes, `TAMAÑO` a `TAMANO`); la lógica de Douglas no se tocó	Todos los casos coincidieron con lo esperado
Pruebas JUnit, README, UML final y casos de la matriz	Nicolás	83 pruebas JUnit **escritas íntegramente por Claude**, README, UML final y los 55 casos	Nicolás ejecutó los casos y llenó los resultados; las pruebas JUnit no se modificaron	Claude las ejecutó con un simulador de JUnit y comprobó que detectan 5 errores puestos a propósito; [mvn test en IntelliJ]
Redacción de este informe	Nicolás	Borrador a partir del registro, el historial de commits y los documentos	[completar: qué se editó]	Cada dato se contrastó con el repositorio y los commits

Pruebas JUnit: no sabíamos escribirlas. Ninguno de los dos sabía cómo se escriben ni qué son las pruebas JUnit; por eso se las pedimos a Claude. Las leímos y entendemos qué comprueba cada una (arman una situación con un mazo de orden fijo, ejecutan una acción y comparan con lo esperado), pero no las escribimos nosotros.

Cuándo la IA se equivocó y cómo lo detectamos. (1) Dijo que "Pedir carta" ya agregaba cartas cuando el botón no estaba conectado: lo notamos al probarlo y lo conectamos. (2) Dio HU04 por completa sin comparar con los criterios 4.3 y 7.5; lo detectamos al revisar el controlador contra cada criterio. (3) Una indicación ambigua dejó métodos pegados dentro de `actualizarVista()`; IntelliJ marcó errores en rojo y se movió la llave. Ver **Anexo, registros 1 y 2**.

Índice de evidencias:

Historial de commits: https://github.com/NicolasRevelo/Mini-proyecto-1-FPOE/commits/main

Matriz de pruebas (55 casos): docs/MATRIZ_PRUEBAS.md

Registro de uso de IA: docs/REGISTRO_IA.md

Pruebas JUnit del modelo: src/test/java/.../modelo

UML inicial y final: docs/UML_final.png

6. Sustentación grabada

Video grupal (MVC, clases, ejecución, una decisión de diseño y una mejora): [enlace]. Video individual de Nicolás (máx. 2 min, pregunta asignada: [pregunta]): [enlace]. Video individual de Douglas: [enlace]. Todos los enlaces con acceso para la docente.

7. Reflexiones individuales

Nicolás Revelo Noreña. Mi aporte fue la vista y el controlador, con apoyo de Claude: la pantalla en FXML, el controlador que lee la Partida y la dibuja, el flujo de apostar, pedir carta, plantarse y turno del computador, las reglas, el cierre con confirmación y la matriz de pruebas. Lo más difícil fue configurar JDK 21 y Maven, y conectar los botones: creí que Pedir carta funcionaba hasta que lo probé y no mostraba nada. Entendí que el controlador no decide las reglas; solo le pregunta a la Partida y dibuja su respuesta. Mi limitación pendiente es que las pruebas JUnit las escribió Claude y aún no sabría hacerlas solo; también falta confirmar con la profesora la definición de Veinte y medio.

Douglas Velez Monginaite. [Reflexión de Douglas, entre 100 y 150 palabras: aporte propio, dificultad, cambio de comprensión y limitación pendiente.]